

CS120 Project Report

Hanrui Zhang (张涵锐), Xiang Li (李想)

Introduction

For those starting from the summer vacation: start from PA2

CS120 project, the Atherneth, requires students to implement the physical layer in PA1, the mac layer in PA2, the ip layer in PA3 and PA4, and the application layer if needed in PA3 and PA4. We strongly recommend students who try to work for it from the summer vacation starting from PA2, where you can simulate your physical layer transmission on file streams. If not, you may lose much time on invalid debugging attempts, since PA1 highly depends on your audio device.

PA1 - Acoustic Link

We started our project from the summer vacation. We believed that the more we've done in the summer, the less we needed to hurry during the semester. However, in the atherneth project, 2 devices are required at least to test for transmission, which is hard for 2 students apart to check the functions. Although we redirected the output to .wav file, which is then used to check the waveform, and used smartphone's speaker to test for receiving, we still spent much time for adjustment after the semester has began.

Audio Output

Before we started our project, Linshu Yang has offered his [Rust ASIO Guide](#), who has highly recommended us to code in Rust. This guide helped us a lot, except for the `OutputStream`, which bothered us in PA2 later.

We spent many days working on the preamble wave. During the summer vacation, we initially thought that any regular signal could be used as the preamble. Based on this assumption, we used a pattern of 1010101010 encoded by a carrier wave, which ended up causing a lot of confusion and left us feeling lost. At first, we believed the issue might lie with the detection program, so we made three different versions of the implementation, hoping to fix the problem. However, nothing changed, and the results were still the same.

It was at this point that we realized the problem might not be with the detection software but with our choice of preamble itself. After this realization, we started searching for a reference solution. That's when we came across the concept of the chirp signal, which turned out to be much more effective in our current situation. Using the chirp significantly improved the results, and one of our biggest obstacles was finally cleared. With this new approach, we were able to receive correct data on occasion, which was a huge step forward.

For the carrier wave, we chose PSK of sine wave for the representation of 0 and 1. However, as the testing work went on, we found that our audio device can only behave well in specific frequencies, which prevented us to achieve the speed requirement using a single frequency carrier. That brought out the OFDM.

To achieve orthogonality, classic OFDM requires the quotient between each carrier frequencies to be a power of 2. According to our calculation, to reach the highest transmission efficiency, the number of carriers must be 1 or 2 when the highest frequency keeps the same. For the latter, the introduction of the lower frequency means more repetition for a single symbol in higher frequency, which enhances the stability. So we chose the classic OFDM with 2 orthogonal carriers.

Device

Device selection played a crucial role in our results. Initially, we started with the PC microphone and the iPhone speaker. While the results weren't perfect, they were stable. However, our TA informed us that the phone's speaker was banned, and we were required to use two PCs instead. So, we tried using two PCs directly, but the outcome was extremely poor. Feeling frustrated, we decided to invest in a new microphone, pairing it with another PC's speaker, hoping this would improve the situation. Sadly, there was no noticeable improvement, and after a few days of troubleshooting, we still hadn't made any progress in solving the problem.

At that point, we were really stuck and didn't know what to do next. Then Lei Huang suggested that the speaker could have a significant impact on the results. He mentioned a woofer we could borrow and try. With this new equipment in hand, we saw some improvement—our accuracy reached around 80%. While this was better, it was still far from meeting the requirements.

Realizing that switching devices could lead to major changes in performance, we borrowed yet another speaker. After making this final change, all of our tests passed without issue. We still vividly remember the day when we simply swapped the speaker, and suddenly, everything fell into place, with all test cases passing successfully.

Framing

As 100% correctness is required for full score of some mandatory tasks, we imported Reed-Solomon ECC to reduce transmission errors. Since the library requires encoding for specified length of bytes, we made our physical frame to fit in these APIs. And as ECC is employed, we didn't implement CRC on our physical frame.

PA2 - Manage Multiple Access

The physical transmission for PA2 is somehow different from one for PA1, as the output stream in [Rust ASIO Guide](#) behaves different in single transmission and multiple transmissions as we need to send a single mac frame at once.

Physical Transmission

In the [Rust ASIO Guide](#), it was suggested to create the `rodio::stream::OutputStream` for multiple uses. However, we encountered an issue where the receiver could not decode the acoustic signal correctly after the first transmission. We spent a significant amount of time debugging to pinpoint the problem.

Initially, we tried creating a new `rodio::stream::OutputStream` each time we transmitted the MAC frame and destroying it after the transmission. While this approach seemed to change the behavior, the problem persisted. The first packet was received correctly, but subsequent packets failed. Upon further investigation, we discovered an unusual issue: when the speaker had no task to

perform, it would emit a constant-frequency noise until the next task started. This noise made detecting the preamble for the next transmission much more difficult than usual. This was a rather strange problem, and we couldn't find any relevant information about it.

Finally, we discovered that if we allowed the output thread to sleep for a short period after the OutputStream ended, the issue was resolved. With this change, the problem disappeared, and we were able to proceed to the next task without further issues.

Since cable transmission is more reliable than the air, we removed the ECC part, and added 1-byte checksum before the whole physical frame. Then, we used Manchester Coding for symbol representation, which only used 2 sample points, bringing more transmission speed.

Mac Frame

To spend less time in transferring non-payload data, we compressed the mac header into 2 bytes: 4 bits for destination mac address, 4 bits for source mac address, 6 bits for frame id, and 2 bits for frame type, after which the payload came.

CSMA

CSMA wasn't a problem for us, since we just needed to design an agreed-mechanism to avoid the collision from both the nodes. Instead, CSMA with Interference brought much trouble for us, since the collision from jammer is somehow unpredictable. We have referred to the parameters in Linshu Yang and Lei Huang's project, but it didn't work for our implementation. We also spent a whole afternoon adjusting our configuration during the code check, but still failed passing task 4.

PA3 & PA4 - To the Internet & Above IP

To pass some specific tasks, we chose Wintun as our virtual network device at the beginning, which made PA3 and PA4 somehow similar.

Virtual Network Device

As wintun offered APIs to process IP packets from the application layer, we can just check the content, and develop a firewall-liked project. We call the packets from the upper layer and sending to the mac layer the "down" packet, and the opposite be the "up" packet.

When the gateway is set to the network device, the OS will send most of the packets to it, and thus a filter is necessary:

- In PA3, we only allow ICMP packets send to the mac layer. For ICMP request, we generate the reply packet in our program to improve the performance.
- In PA4, besides ICMP, DNS for websites in the tasks, and packets corresponding to the ip of these websites were also allowed.

Task 2 in PA4 required us to specify the sequence number of our TCP connection. As it would spend much time to do so on our project, which required to detect the TCP flags, change the sequence number of the down packet (also record it), set the sequence number of the up packet to the original one, (and Hanrui was busy reviewing for other courses, also determined to be back home early), we implemented a simpler one: added a fixed number to the down packet, and subtracted it to the up packet. Of course checksum in TCP header was still needed to modify.

Router

As our project only worked as a firewall, we needed to modify the route table on Windows to achieve Internet tasks.

Windows is capable of forwarding IP packets whose destination is not itself, but it struggles when one interface receives packets intended for another interface. To address this, we used the New-NetNat command to create a NAT device on the Acoustic interface and enabled packet forwarding for all devices by running Set-NetIPInterface -Forwarding Enable. With this setup, Acoustic worked well, and we were able to access the external web without issues.

However, we encountered a problem with the forwarding between the hotspot and Acoustic. Our suspicion was that the issue stemmed from the shared device between the hotspot and Ethernet, which limited the forwarding capabilities due to the narrow interface configuration. Since we didn't need to implement both NAT and access to the outer web at the same time, we decided to replace the shared device with Acoustic. This allowed the hotspot and Acoustic to successfully connect with each other.

With this adjustment, we were able to meet all the requirements in the normal parts of PA3 and PA4.